LECTURE 10 – Introduction to Turing Machines

1. What Is a Turing Machine?

A **Turing Machine (TM)** is a very simple *imaginary computer model* used in Computer Science to understand how computation works.

Even though it is simple, it is powerful enough to simulate anything a modern computer can do.

A TM has three main parts:

1. A Tape

- Works like a very long sheet of memory.
- Stores symbols like 0, 1, or blank (_).

2. A Tape Head

- Reads a symbol from the tape
- Can write a new symbol
- Can move **left** or **right**

3. A Set of Rules

These rules tell the machine what to do based on what it reads.

---

2. How Does a Turing Machine Work?

The TM repeats the following steps:

1. **Reads** the symbol under the head
2. **Decides** what to do according to its rules
3. **Writes** a new symbol (or keeps the same)
4. **Moves** left or right
5. **Changes state** if needed

It continues until:

- It reaches a **final state** → meaning the input is **accepted**, or
- It loops forever (keeps working without stopping)

3. What Does “Accept” Mean?

A TM **accepts** an input if:

- It reaches a final (accepting) state
- And **stops**

If it stops in a non-final state → it **rejects** the input.

---

4. Example of What a TM Can Do

A TM can check simple patterns, like:

- Does the string **000111** have the same number of 0s and 1s?
- Does the string follow a certain format?
- Convert 0s into 1s
- Move through the tape and search for something

This helps us understand the limits of computation.

---

5. Why Do We Study Turing Machines?

Because they help us answer important questions:

- **What can computers solve?**
- **What problems are impossible to solve?**
- **How powerful is computation?**

Turing Machines are the foundation of theoretical computer science.

LECTURE 11 – Decidability & Undecidability

1. Not All Problems Can Be Solved

Some problems can always be solved by a machine, but others **cannot**.

We divide problems (or languages) into three main types:

---

2. Recursive (Decidable) Languages

A machine working on a recursive language:

- **Always stops**
- Gives a clear **YES** or **NO** answer
- Never loops forever

These are "easy" in the sense that computers can always solve them.

---

3. Recursively Enumerable (RE) Languages

A machine working on an RE language:

- **Stops ONLY when the answer is YES**
- Might loop forever when the answer is NO

These languages are harder.

---

4. Non-RE Languages

Languages that:

- **No machine can recognize**
- Impossible for computers to handle
- Too complex or too large for any TM

5. The Halting Problem

A famous question:

Can a machine always decide whether another program will stop?

The answer is **No** — this is called the **Halting Problem**, and it is **undecidable**.

No machine can always determine if another machine will stop or run forever.

This was proven by Alan Turing.

---

6. Why Are Some Problems Undecidable?

Because:

- There are **countably many** Turing Machines (like counting whole numbers)
- But there are **uncountably many** possible languages (far more)

That means some problems are simply **too big** for machines to handle.

---

7. Reductions

A reduction means turning one problem into another.

We use it to show:

- If problem A is unsolvable
- And we can convert A → B
- Then B must also be unsolvable

This is how we prove many problems are **undecidable**.

SUMMARY (Both Lectures Together)

You learned:

Lecture 10

- What a Turing Machine is
- How it reads, writes, and moves
- How it accepts an input
- Why TMs matter in computer science

Lecture 11

- Difference between Recursive, RE, and Non-RE languages
- Why some problems are unsolvable
- The Halting Problem
- How reductions prove undecidability

PRACTICAL QUESTION 1 — Tape Simulation (4 Steps)

A Turing Machine receives the tape:

1 0 1 0 _ _

The head starts on the **first 1**.

The machine follows these rules:

1. If reading **1**, change it to **X** and move right
2. If reading **0**, change it to **Y** and move right
3. If reading **X** or **Y**, move right without changing anything

Task:

Simulate the **first FOUR steps** of the machine.
After each step, write the updated tape and the new head position.

ANSWER (Step-by-Step)

Initial:
1 0 1 0 _ _ (Head on first 1)

Step 1: 1 → X, move right
X 0 1 0 _ _ (Head on 0)

Step 2: 0 → Y, move right
X Y 1 0 _ _ (Head on 1)

Step 3: 1 → X, move right
X Y X 0 _ _ (Head on 0)

Step 4: 0 → Y, move right
X Y X Y _ _ (Head on blank)

Final tape: X Y X Y _ _

PRACTICAL QUESTION 2 — Multi-Step Tape Execution (3 Steps)

A Turing Machine receives the tape:

1 0 0 1 0 _ _

The head starts on the **first 1**.

The machine follows these rules:

1. If reading **1**, change it to **X**, move right
2. If reading **0**, change it to **Y**, move right
3. If reading **X** or **Y**, move right

Task:

Simulate the **first THREE steps** of the machine.
Write the tape and head position after each step.

ANSWER (Step-by-Step)

Initial:
1 0 0 1 0 _ _ (Head on 1)

Step 1: 1 → X, move right
X 0 0 1 0 _ _ (Head on 0)

Step 2: 0 → Y, move right
X Y 0 1 0 _ _ (Head on next 0)

Step 3: 0 → Y, move right
X Y Y 1 0 _ _ (Head on 1)

Final tape: X Y Y 1 0 _ _

Bidirectional Movement & Marking

A Turing Machine receives the tape:

1 0 1 0 1 _ _

The head starts on the **first symbol** (the first 1).

The machine follows these rules:

1. **If reading 1 → change to X, move right**
2. **If reading 0 → change to Y, move left**
3. **If reading X or Y → move right**
4. **If reading _ → stop**

Task:

Simulate the machine **step-by-step** until it stops.
After each step, write:

- The updated tape
- The head's new position
- The rule applied

When the machine stops, write:

- **Total number of steps**
- **Final tape configuration**
- **Final head position**

MODEL ANSWER (Step-by-Step)

Initial Tape:

1 0 1 0 1 _ _
Head on **1**

Step 1:

Reads **1** → Rule 1 → write **X**, move right
Tape: X 0 1 0 1 _ _
Head on the first **0**

Step 2:

Reads **0** → Rule 2 → write **Y**, move left
Tape: X Y 1 0 1 _ _
Head moves left onto **X**

Step 3:

Reads **X** → Rule 3 → move right
Tape unchanged: X Y 1 0 1 _ _
Head on **Y**

Step 4:

Reads **Y** → Rule 3 → move right
Tape unchanged: X Y 1 0 1 _ _
Head on **1**

Step 5:

Reads **1** → Rule 1 → write **X**, move right
Tape: X Y X 0 1 _ _
Head on **0**

Step 6:

Reads **0** → Rule 2 → write **Y**, move left
Tape: X Y X Y 1 _ _
Head on **X**

Step 7:

Reads **X** → Rule 3 → move right
Tape unchanged: X Y X Y 1 _ _
Head on **Y**

Step 8:

Reads **Y** → Rule 3 → move right
Tape unchanged: X Y X Y 1 _ _
Head on **1**

Step 9:

Reads **1** → Rule 1 → write **X**, move right
Tape: X Y X Y X _ _
Head on _

Step 10:

Reads _ → Rule 4 → **STOP**

FINAL RESULTS

- **Total steps:** 10
- **Final tape:**
  X Y X Y X _ _
- **Head stopped on:** the **first blank symbol**